

GNNs and KG Embeddings

for Prediction Tasks on Real-World Graphs

Adrian Patricio & Oluwanifemi Olajuyigbe

MGMA Mini Project

Dataset 1

ArXiv GR-QC

Collaboration Network

Link Prediction with GNNs

Dataset 2

BioDecagon

Polypharmacy Graph

Link Prediction with KG Embeddings

Part 1 Recap

Centrality Measures and Community Detection Algorithms for Real-world Graphs

Focus

Analyzing community structures in the **Facebook** (social) and **Email-Eu-Core** (organizational) networks.

Core Algorithms

Compared traditional density-based algorithms like Louvain, LPA, Walktrap – a little bit of experimentation with Spectral and Node2Vec

Dataset 1: ArXiv GR-QC

ArXiv GR-QC Dataset at a Glance

5,242

Author nodes

14,496

Co-author edges

Undirected

Graph Type

10yrs

1993 – 2003

What it is

A collaboration network of scientists who co-authored papers on arXiv's General Relativity & Quantum Cosmology section. Each node is an author, and each edge represents co-authorship.

Our Task

Link Prediction: given the graph, predict whether two authors will collaborate.

Evaluation: Global metrics (AUC, AP) and ranking metrics (MRR, Hits@K) against 99 random negatives.

Splitting & Leakage Prevention

RandomLinkSplit



Divides all edges into train (80%) / val (5%) / test (10%). A disjoint split ensures test edges are never seen during training message-passing.

Balanced negatives



Equal number of fake edges added automatically → 1:1 positive:negative ratio for fair binary classification.

No data leakage



Node features computed **ONLY** on the training graph. Using the full graph would leak test edge info into features.

No overlap verified



Random seed fixed. Train / val / test pairs verified to never share any edge. Test set stays completely hidden.

Exact Split Numbers

5,242

Nodes

3

Features per node

24,646

Training edges
(message-passing)

1,448

Val pairs
(724 pos · 724 neg)

2,898

Test pairs
(1,449 pos · 1,449 neg)

Training supervision pairs: 24,646

12,323 positive · 12,323 negative · 1:1 ratio

No overlap between splits

Train / val / test pairs verified disjoint ✓

Message-passing edges $\neq$ supervision pairs — the model sees 24,646 edges for neighbourhood aggregation, and is trained to score 24,646 separate labelled pairs (pos + neg).

3 Structural Node Features

GR-QC has no node attributes, so we engineer three features from the graph. All features are computed on the training graph only and normalized to $[0,1]$.

Degree

$$\text{deg}(v) / \text{max_degree}$$

Number of co-authors, normalized.

→ High degree authors are far more likely to form new collaborations.

Clustering Coefficient

$$2 \cdot |e_{uw}| / k(k-1)$$

Fraction of a node's neighbors who are also connected.

→ High clustering → tight research clique. Members likely to collaborate.

PageRank

$$\text{PR}(v) = (1-d)/N + d \cdot \sum \text{PR}(u)/L(u)$$

Global importance in collaboration network.

→ Important authors have wide collaborative reach, boosting link probability.

Three GNN Encoders

2-layer message-passing networks. Layer 1: ReLU + Dropout(0.5). Layer 2: no activation (raw embeddings for decoder).

GCN

Symmetric normalisation

$$H^{(1+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(1)} W^{(1)})$$

Adds self-loops ($\tilde{A}=A+I$).
Aggregates all neighbours
weighted by degree.

GraphSAGE

Sample & Aggregate

$$h_v = \sigma(W \cdot \text{MEAN}(\{h_u \in N(v)\}) + B \cdot h_v)$$

Aggregates neighbor features
with mean pooling.
Separates ego embedding from
neighborhood – more flexible
than GCN.

GAT

Attention-weighted

$$h_v = \sigma(\sum_u \alpha_{vu} \cdot W \cdot h_u) \quad [4 \text{ heads}]$$

Learns attention weights –
which neighbours matter most.
4 heads in layer 1
(concatenated), 1 head in layer 2
(averaged).

Two Decoders

Dot Product

$$\text{score} = \mathbf{z}_u^T \cdot \mathbf{z}_v$$

Assumes collaborating authors have similar embeddings. Simple dot-product similarity.

⚠ Misses non-linear collaboration patterns

Faster to train · fewer parameters

MLP Decoder

$[\mathbf{z}_u \parallel \mathbf{z}_v] \rightarrow$
Linear→ReLU→Dropout→Linear→Score

Concatenates both embeddings. Learns non-linear, complex interaction patterns between node pairs.

✓ Captures non-linear collaboration dynamics

All models: logits output → BCEWithLogitsLoss

Training Pipeline

1 Loss

Binary Cross-Entropy

$$L = -(1/N) \sum [y_i \log(\hat{p}_i) + (1-y_i) \log(1-\hat{p}_i)]$$



2 Optimizer

Adam optimizer
 weight decay 5e-4



3 Hyperparameter Tuning

LR $\in \{0.005, 0.01\}$
 hidden_dim $\in \{64, 128\}$
 out_dim $\in \{32, 64\}$

Best config selected by
 validation AUC



4 Early Stop

Patience = 40 epochs
 Best model weights
 restored when val AUC
 stops improving

Evaluation Metrics

measures how precisely a model ranks positive examples near the top.

AUC-ROC

Model's ability to rank
 positive examples higher
 than negative ones.

Avg Precision

Precision-recall: ability to
 precisely rank positive
 examples near the top.

MRR

Rank of true edge vs 99
 negatives

Hits@K

Top-K accuracy (K=1,10,50)

Test Set Performance

Model	AUC-ROC	Avg Prec.	MRR	Hits@1	Hits@10	Hits@50	Time (s)
GraphSAGE + MLP ★	0.9096	0.9174	0.3868	0.2636	0.6025	0.9289	25.8s
GAT + MLP ★★	0.9075	0.9171	0.4353	0.3071	0.6625	0.9317	36.9s
GCN + MLP	0.8990	0.9075	0.3516	0.2153	0.5997	0.8999	29.8s
GraphSAGE + Dot ⚡	0.8362	0.8635	0.2761	0.1477	0.5342	0.8592	15.0s
GAT + Dot	0.8492	0.8648	0.3127	0.1698	0.5825	0.8951	29.0s
GCN + Dot	0.8306	0.8608	0.2462	0.1166	0.5155	0.8578	21.3s

★ Best global AUC ★★ Best ranking (MRR & Hits@1) MLP beats Dot in every single row

Efficiency vs Performance: GraphSAGE provides the best balance of speed and performance. GraphSAGE+dot is fastest while GAT+MLP is slowest but wins on ranking. MLP decoders add training time due to extra backprop, but worth it for accuracy.

What the Results Tell Us

① MLP always beats Dot

This suggests collaboration is non-linear. The MLP learns complex patterns dot-product similarity cannot capture.

② GAT = ranking champion

MRR 0.4353 · Hits@1 0.307. Attention focuses on the exact right co-authors in dense cliques. . Best model when the goal is identifying the single most likely collaborator.

③ GraphSAGE = global winner

Best AUC 0.9096. Most robust at globally separating real from fake edges. Mean aggregation generalises robustly across degree-skewed author distributions.

④ Hits@50 ≈ 93% vs Hits@1 ≈ 31%

Models find the right research group easily (placing true collaborator in top half of candidate 93% of the time) but picking the exact co-author from a dense clique is hard.

Dataset 2: Decagon

BioDecagon Dataset at a Glance

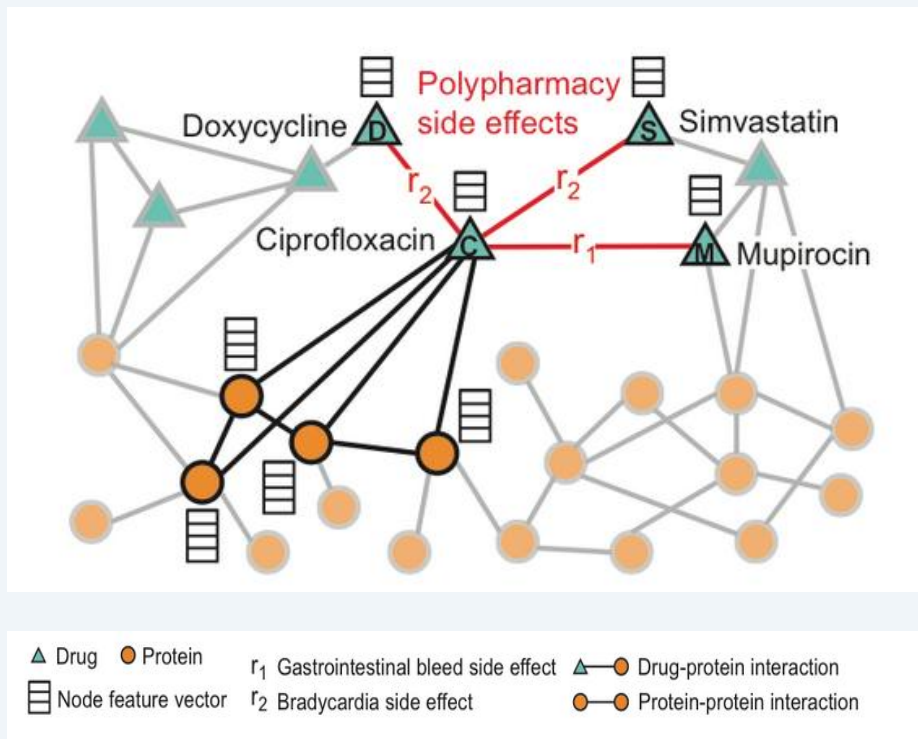
NODES

645

Active drugs

19K+

Proteins



EDGES

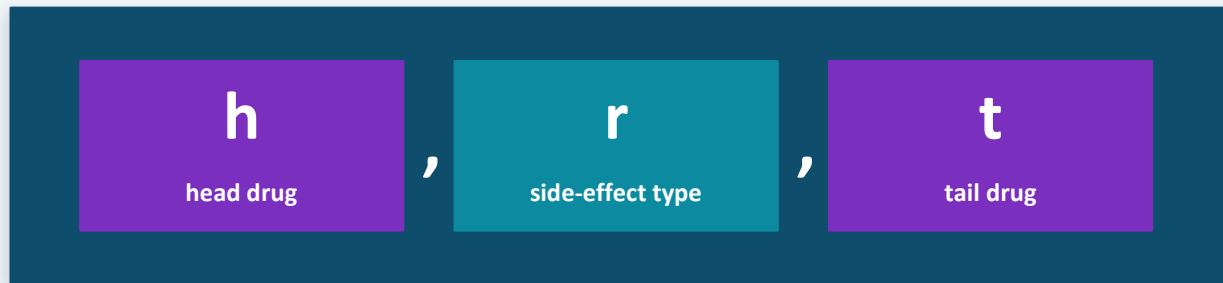
964

Side-effect types

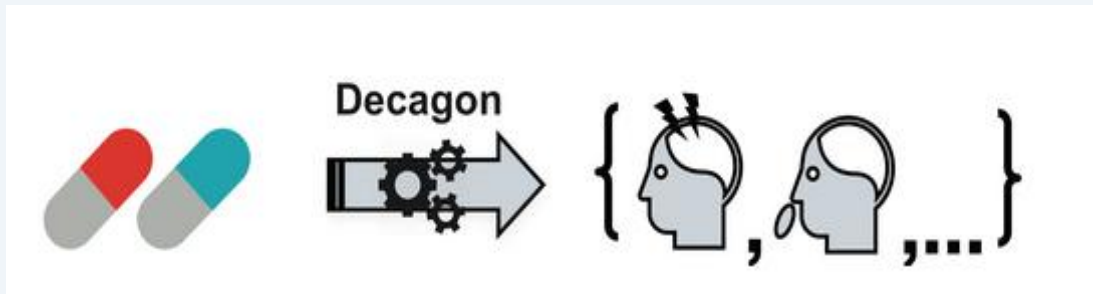
+ DRUG – PROTEIN TARGET

Subgraph for experiments: TOP_K_SE = 50 most frequent side-effect types → ~975K edges, ~645 active drugs. Rare SE types lack sufficient training examples for reliable learning.

Triple Representation



80/10/10 train/val/test split. Negatives by corrupting the tail.



What the Data Tells Us



Long-tailed SE distribution

A few common side effects appear in thousands of pairs and most are rare.
→ Subsample top-50 SE types for reliable training.



Strong 1-to-N patterns

Avg 16 tails per head. TranE forces all 16 drugs to the same region → collapses them.
Handle this via projections or complex embeddings.



Skewed drug degrees

Most drugs have degree ~1. A few hubs dominate.
→ Hub-heavy graphs hurt full-neighbourhood aggregation (motivates GraphSAGE).



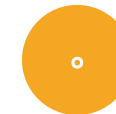
No reflexivity (0 self-loops)

Polypharmacy involves two different drugs.
Dataset is well-formed. No model needs to handle self-interactions.



Semantically Symmetric

Drug A+B = Drug B+A basically – they cause the same side effect whatever the order
→ DistMult's symmetric scoring aligns with the data.



82.9% composition rate

Two drugs connected via intermediary are 83% likely to also be directly connected.
→ Favours RotatE, naturally models relation composition

Link Prediction

$(h, r, ?)$

Given Drug A + nausea,
which drug causes nausea
when combined with Drug A?

Keep (h, r) fixed. **Score all 645 drugs** as candidate tails. Rank by score. Find rank of true tail.

Clinical use: drug substitution, drug detection

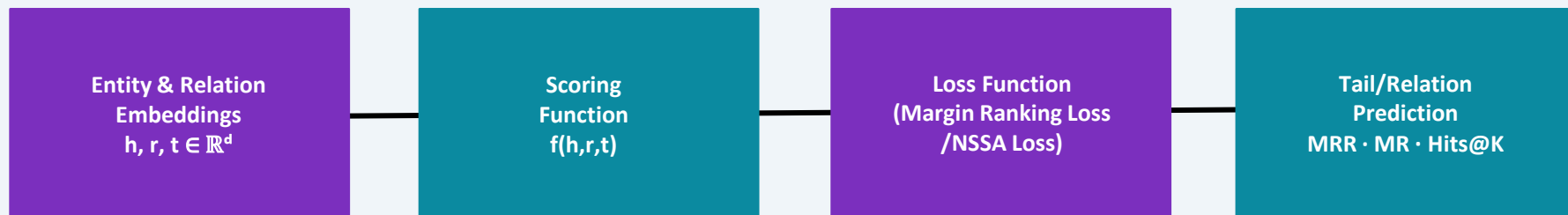
Relation Prediction

$(h, ?, t)$

Given Drug A + Drug B
taken together, **which**
side effect will they cause?

Keep (h, t) fixed. **Score all 50 SE types** as candidate relations. Rank by score.

Clinical use: monitoring the side effects



Loss Function

Margin Ranking Loss

$$L = \sum \max(0, \gamma + f_{\text{neg}} - f_{\text{pos}})$$

> Loss pushes positive above negative by margin γ . This directly optimizes the ranking objective — same signal as MRR/Hits@K.

NSSA Loss:

$$L = -\log \sigma(s+) - \sum \alpha_k \log \sigma(-sk-)$$

> Similarly, this pushes positive above negative by margin γ but with weights for hard samples. This is used for **RotaTe**,

Model	Scoring $f(h,r,t)$	Key Idea
TransE	$- h + r - t $	Translation; breaks for 1-to-N
TransH	$- h_{\perp} + r - t_{\perp} $	Hyperplane projection; handles 1-to-N better
TransR	$- h \cdot M_r + r - t \cdot M_r $	Relation-specific projection matrix
DistMult	$h^T \text{diag}(r) t$	Symmetric bilinear; natural for DDI
ComplEx	$\text{Re}(h^T \text{diag}(r) \bar{t})$	Complex domain; handles asymmetry
RotatE	$- h \circ r - t $	Rotation in complex space; rich expressiveness

KG Embedding Results

TAIL (h, r, ?) ~645 drugs			
Model	MRR	Hits@1	Hits@10
TransE	0.0373	0.0007	0.0889
TransH	0.0388	0.0001	0.0944
TransR	0.0392	0.0069	0.0804
DistMult	0.0315	0.0063	0.0594
Complex ★	0.0489 ★	0.0113 ★	0.1024 ★
RotatE	0.0424	0.0100	0.0848

Complex leads tail prediction.

Complex embeddings capture patterns more expressively. Lower MRR overall due to . Lower MRR overall due to ~645 entity candidates. Larger candidate set warrants more expressive models.

RELATION (h, ?, t) 50 SE types			
Model	MRR	Hits@1	Hits@10
TransE	0.1772	0.061	0.439
TransH	0.2139	0.095	0.507
TransR	0.2054	0.083	0.521
DistMult ★	0.2479 ★	0.120 ★	0.560 ★
CompLex	0.2344	0.109	0.540
RotatE	0.1984	0.087	0.447

DistMult leads relation prediction.

Edges are symmetric (Drug A + Drug B = Drug B + Drug A). DistMult's symmetric scoring is a perfect fit. Relation prediction easier (50 candidates vs 645). Probably smaller candidate set benefits simpler models.

What the Results Tell Us

① Translational Model Performance

TransE performs poorly, but generally similar performance for all 3. Tradeoff between 1-N handling for TransE, composition handling for TransH and moderate performance for TransR.

② DistMult Performance

Interestingly, DistMult performed worse for tail, and best for link prediction. It probably is too simple to handle distinguishing the different nodes, but it is probably simple enough to handle relations.

③ ComplEx/RotAte

Complex-valued scoring captures richer interactions than real-valued models. RotAte treats relations as rotations, which captures composition effectively, and is more expressive

④ Relation Search Space

The model must pick 1 correct drug out of ~**645 candidates** for tail prediction = more complex ranking, and 1 side effect out of **50 labels** for relation prediction = more simple

R-GCN Architecture and Pipeline

R-GCN Encoder (2-layer)

Layer: $h_i^{(l+1)} = \sigma(\sum_r \sum_{j \in N_r(i)} (1/|N_r(i)|) W_r^{(l)} h_j^{(l)} + W_o^{(l)} h_i^{(l)})$

- **One weight matrix W_r per relation type.**
- Self-connection W_o preserves self-features.
- ReLU between layers, no activation at output.
- **Input:** random 64-d features (no real drug properties).

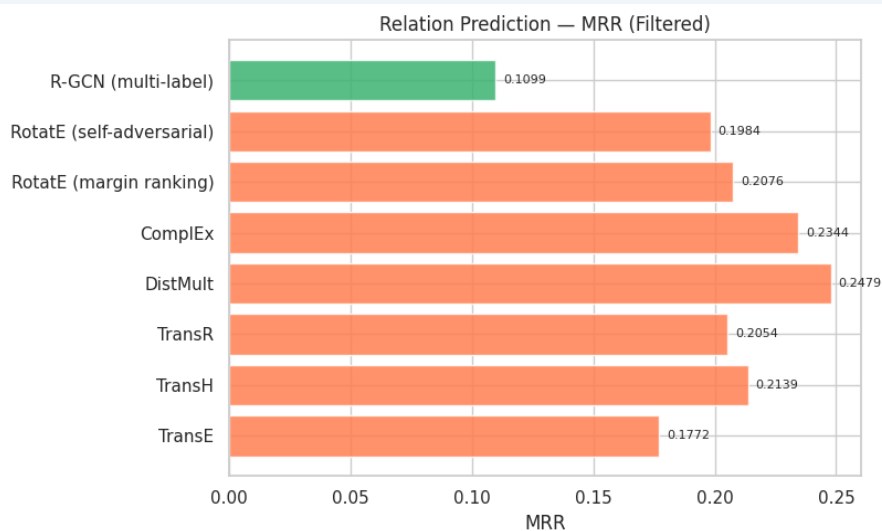
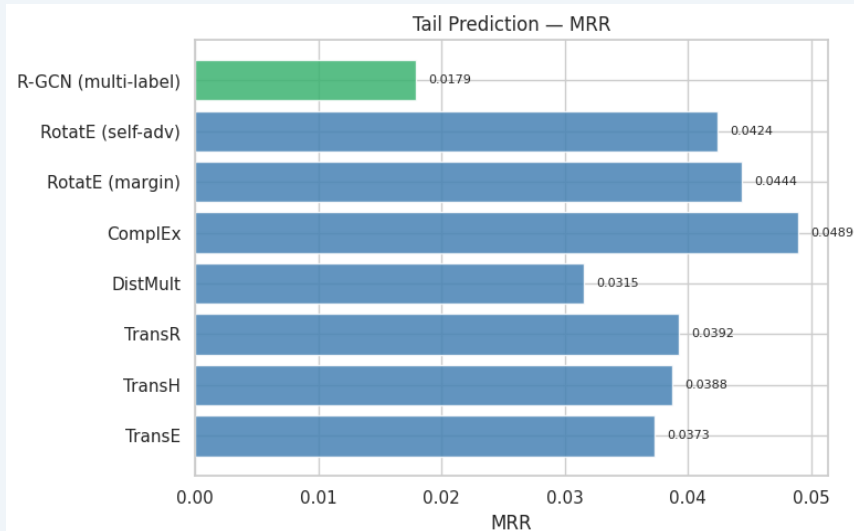
Decoder

- $\text{score}(\text{Drug_A}, r, \text{Drug_B}) = z_A^T \cdot \text{diag}(r_vec) \cdot z_B$
- For each of the 50 SE types, a relation vector is learned.
- **Training:** Multi-label BCE — predict which of 50 SEs are active for each drug pair.
- **Evaluation:** same MRR/Hits@K protocol as KG models.

Why R-GCN?

Standard GNNs (GCN/SAGE/GAT) ignore edge types. They aggregate neighbors equally. R-GCN uses separate weight matrices per relation type, allowing it to distinguish 'Drug A connected via nausea' from 'Drug A connected via bleeding'. This is critical for multi-relational biomedical graphs.

R-GCN Results



Why R-GCN Performed Poorly?

- R-GCN uses multi-label BCE (classification). The model learns to classify, not to rank. Evaluation on a ranking metric penalizes this mismatch.
- R-GCN derives embeddings from shared GNN weights applied to random input features. All drug representations share the same weight matrix which gives less capacity for individual drug identity for added scalability.
- R-GCN Implementation is purely structural, no node features were used which would have improved this probably and taken advantage of message passing.
- R-GCN shines for inductive learning. It CAN generalize to unseen drugs (new nodes at inference). KG embedding models cannot.

Questions?